```
!pip -q install opencv-python-headless

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from google.colab.patches import cv2_imshow
import os

print("OpenCV version:", cv2.__version__)
```

```
OpenCV version: 4.14.0
```

```
uploaded = files.upload()

image_name = next(iter(uploaded.keys()))
image = cv2.imread(image_name)

if image is None:
    raise ValueError("The uploaded file is not a valid image.")

print("Image loaded successfully:", image_name)
print("Image shape:", image.shape)
```

```
Choose files Massinhas pra gente.jpeg
Massinhas pra gente.jpeg(image/jpeg) - 50680 bytes, last modified: 20/08/2026 - 100% done
Saving Massinhas pra gente.jpeg to Massinhas pra gente.jpeg
Image loaded successfully: Massinhas pra gente.jpeg
Image shape: (736, 736, 3)
```

```
print("Display using OpenCV:")
cv2_imshow(image)

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Display using OpenCV:

Original Image

```
height, width = image.shape[:2]

if image.ndim == 2:
    channels = 1
else:
    channels = image.shape[2]

print("Width:", width, "pixels")
print("Height:", height, "pixels")
print("Resolution:", f"{width} x {height}")
print("Number of channels:", channels)
print("Data type:", image.dtype)
print("Total pixels:", height * width)
```

```
Width: 736 pixels
Height: 736 pixels
Resolution: 736 x 736
Number of channels: 3
Data type: uint8
Total pixels: 541696
```

```
jpeg_name = "processed_image.jpg"
png_name = "processed_image.png"

cv2.imwrite(jpeg_name, image)
cv2.imwrite(png_name, image)

print("JPEG saved:", jpeg_name)
print("PNG saved:", png_name)

print("JPEG size:", os.path.getsize(jpeg_name), "bytes")
print("PNG size:", os.path.getsize(png_name), "bytes")
```

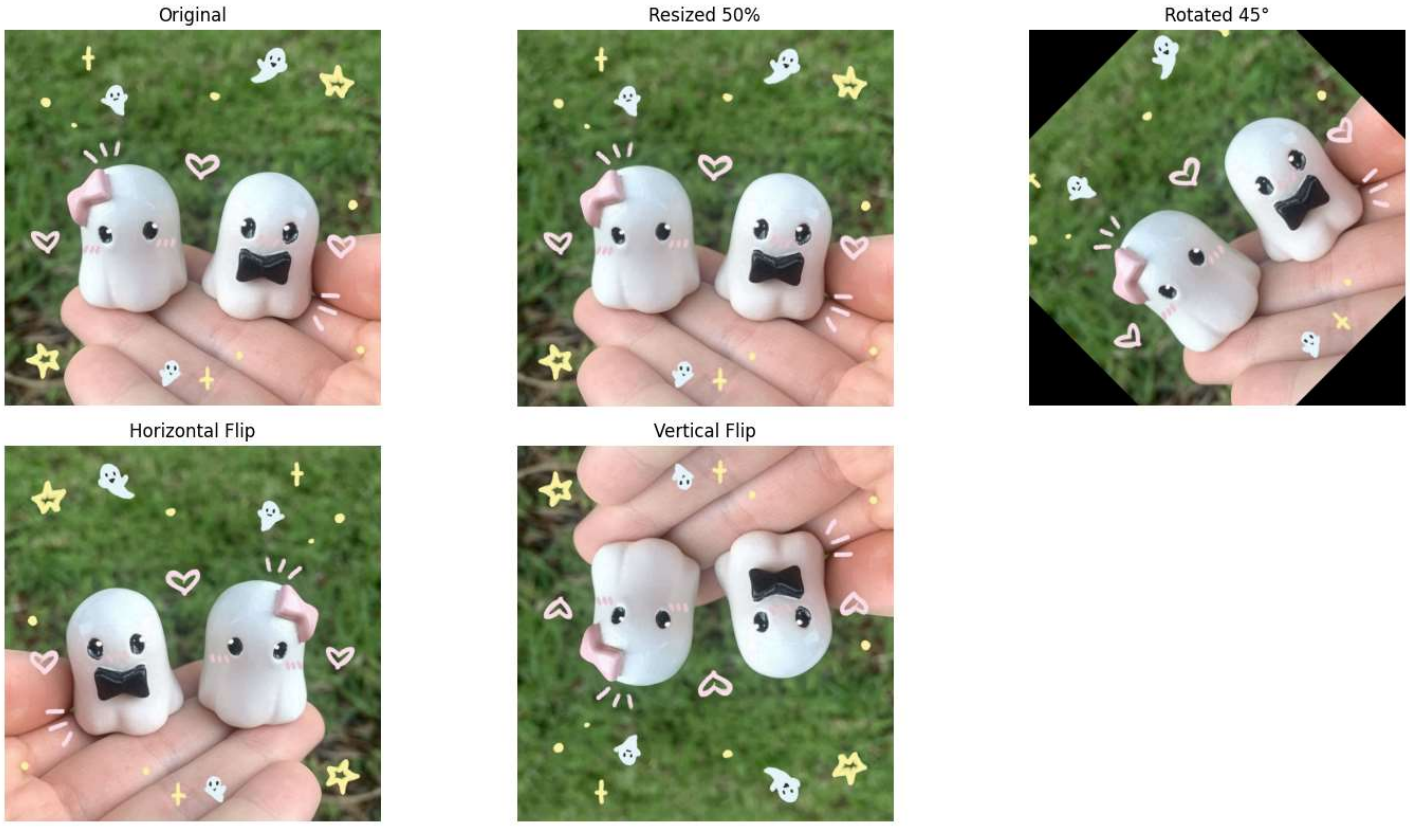
```
JPEG saved: processed_image.jpg
PNG saved: processed_image.png
JPEG size: 92157 bytes
PNG size: 619529 bytes
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.title("LAB")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Grayscale shape:", gray.shape)
print("HSV shape:", hsv.shape)
print("LAB shape:", lab.shape)
```


```
Grayscale shape: (736, 736)
HSV shape: (736, 736, 3)
LAB shape: (736, 736, 3)
```

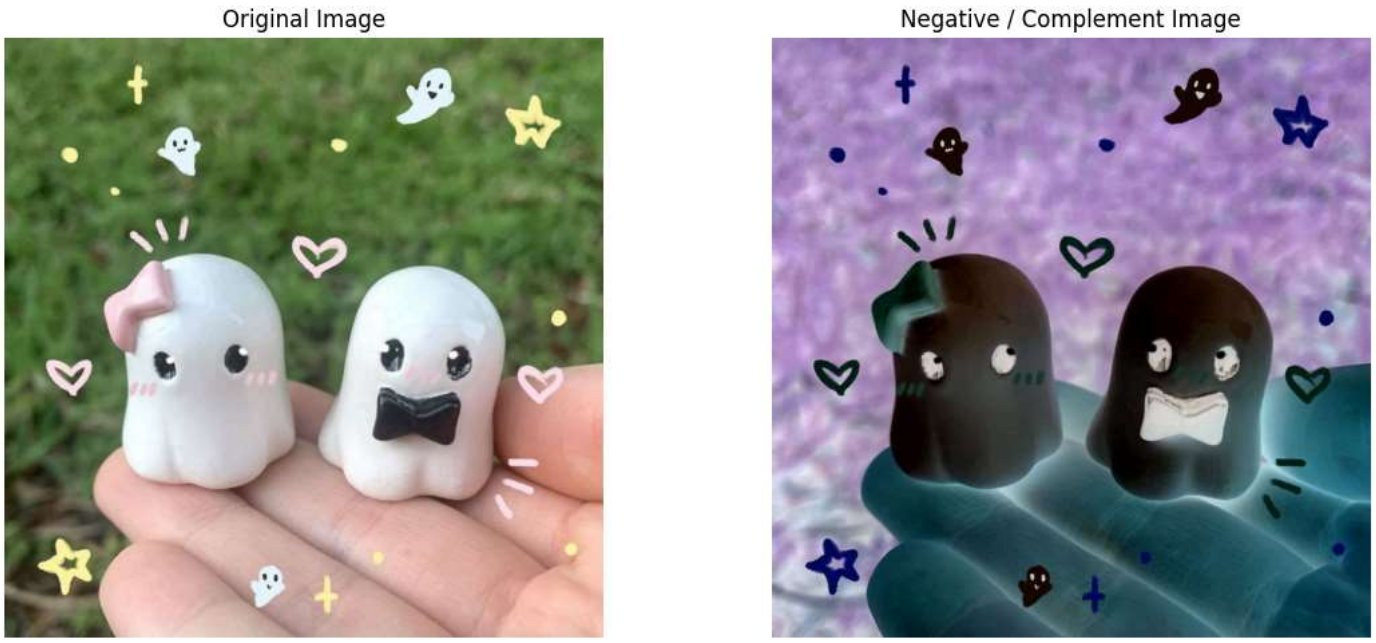
```
# Resize the image to 50% of its original size
resized = cv2.resize(
    image,
    None,
    fx=0.5,
    fy=0.5,
    interpolation=cv2.INTER_AREA
)

# Rotate the image by 45 degrees
height, width = image.shape[:2]
center = (width // 2, height // 2)

rotation_matrix = cv2.getRotationMatrix2D(
    center,
    45,
    1.0
)
```

```
rotated = cv2.warpAffine(
    image,
    rotation_matrix,
    (width, height)
)

# Horizontal flip
horizontal_flip = cv2.flip(image, 1)

# Vertical flip
vertical_flip = cv2.flip(image, 0)

# Display all results
images = [
    (image_rgb, "Original"),
    (cv2.cvtColor(resized, cv2.COLOR_BGR2RGB), "Resized 50%"),
    (cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB), "Rotated 45°"),
    (cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB), "Horizontal Flip"),
    (cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB), "Vertical Flip")
]

plt.figure(figsize=(15, 8))

for i, (img, title) in enumerate(images, 1):
    plt.subplot(2, 3, i)
    plt.imshow(img)
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# Create the negative (complement) image
negative = 255 - image

# Convert negative image from BGR to RGB for Matplotlib
negative_rgb = cv2.cvtColor(negative, cv2.COLOR_BGR2RGB)

# Display original and negative images
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(negative_rgb)
plt.title("Negative / Complement Image")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```




```
# Get image dimensions
height, width = image.shape[:2]

# Define a central ROI covering 50% of the image
roi_width = width // 2
roi_height = height // 2

x_start = (width - roi_width) // 2
y_start = (height - roi_height) // 2

# Crop the ROI
roi = image[
    y_start:y_start + roi_height,
    x_start:x_start + roi_width
]

print("ROI shape:", roi.shape)
print(
    "ROI position:",
    f"x={x_start}, y={y_start}, "
    f"width={roi_width}, height={roi_height}"
)

# Convert ROI to RGB for displaying with Matplotlib
roi_rgb = cv2.cvtColor(roi, cv2.COLOR_BGR2RGB)

# Display original and ROI
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(roi_rgb)
plt.title("Region of Interest (ROI)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
ROI shape: (368, 368, 3)
ROI position: x=184, y=184, width=368, height=368
```